

# SDD — Módulo 5: Métodos de Ordenamiento

## 1. Información general del proyecto

**Nombre del proyecto:** Visualizador Web de Estructuras de Datos

**Módulo:** Métodos de Ordenamiento

**Versión:** 0.1.0

**Estado:** Especificación inicial para implementación del nuevo módulo

**Tecnologías:** Python, Flask, HTML5, CSS3, JavaScript, Jinja2, C didáctico

**Tipo de proyecto:** Aplicación web didáctica

**Público objetivo:** Estudiantes, docentes y monitores de estructuras de datos y algoritmos

### Descripción general

El módulo de métodos de ordenamiento permitirá a los estudiantes comprender, ejecutar y visualizar algoritmos clásicos de ordenamiento sobre arreglos de datos. El usuario podrá ingresar una lista de valores, seleccionar un método de ordenamiento, ejecutar el algoritmo en modo rápido o paso a paso y observar las comparaciones, intercambios, posiciones activas y estado final del arreglo.

Este módulo debe integrarse con la arquitectura general del visualizador y conservar las mismas especificaciones de calidad, documentación, separación de capas, ayudas didácticas, trazabilidad, pruebas y criterios de aceptación definidos para los módulos anteriores.

El proyecto general ya define una visión didáctica basada en visualización interactiva, interpretación gráfica de código C, simulación paso a paso, retroalimentación inmediata y coherencia entre código, visualización y estado.

### Objetivo del módulo

Diseñar e implementar un módulo web didáctico que permita visualizar métodos de ordenamiento, explicar su funcionamiento paso a paso, sincronizar la ejecución con código C/pseudocódigo y comparar sus características principales.

### Alcance inicial

La primera versión del módulo debe incluir:

- Creación de un arreglo de valores.
- Carga manual de valores por el usuario.
- Generación opcional de arreglos aleatorios didácticos.
- Validación de entradas numéricas.
- Visualización del arreglo como barras, celdas o tarjetas.
- Ejecución rápida del algoritmo seleccionado.
- Ejecución paso a paso.
- Resaltado de comparaciones.
- Resaltado de intercambios.
- Resaltado de posiciones ordenadas.
- Conteo de comparaciones.
- Conteo de intercambios o movimientos.

- Historial de operaciones.
- Código C asociado cuando exista mapeo disponible.
- Pseudocódigo como fallback cuando no exista mapeo C.
- Ayuda didáctica general del módulo.
- Ayuda por algoritmo.

### **Algoritmos candidatos para la primera versión**

Los algoritmos exactos deben confirmarse analizando los archivos C disponibles para este módulo. Si están presentes en el TAD de ordenamiento, se deben priorizar:

- Burbuja.
- Selección.
- Inserción.
- Shell Sort.
- Merge Sort.
- Quick Sort.
- Heap Sort.

Si algún algoritmo no existe en los archivos C suministrados, el agente desarrollador debe marcarlo como pendiente y no inventar implementación.

### **Fuera de alcance para esta fase**

- Ordenamiento externo en disco.
- Ordenamiento distribuido.
- Ordenamiento paralelo.
- Persistencia en base de datos.
- Evaluación automática de estudiantes.
- Comparación estadística avanzada.
- Algoritmos no presentes en el TAD suministrado, salvo como pendientes futuros.

## **2. Visión del sistema**

### **Problema que resuelve**

Los métodos de ordenamiento suelen enseñarse con pseudocódigo o tablas estáticas. Sin una visualización paso a paso, los estudiantes pueden tener dificultades para comprender comparaciones, intercambios, pivotes, particiones, fusiones, subarreglos y condiciones de finalización.

El módulo busca transformar estos procesos en una experiencia visual donde cada paso del algoritmo sea observable y comprensible.

### **Usuarios objetivo**

- Estudiantes de estructuras de datos.
- Estudiantes de algoritmos.
- Docentes de programación.
- Monitores académicos.
- Desarrolladores educativos.

## Beneficios esperados

- Comprensión visual de algoritmos de ordenamiento.
- Identificación de comparaciones e intercambios.
- Comprensión de complejidad mediante contadores didácticos.
- Sincronización entre código C y animación.
- Comparación conceptual entre algoritmos simples y avanzados.
- Integración con el sistema de sesiones e historial del visualizador.
- Preparación para futuras implementaciones o variantes en C.

## Contexto de uso

El módulo se usará desde un navegador web dentro de la misma aplicación Flask del proyecto principal.

## 3. Requisitos funcionales

| ID       | Requisito                 | Descripción                                                                       | Prioridad | Actor      |
|----------|---------------------------|-----------------------------------------------------------------------------------|-----------|------------|
| RF-O-001 | Acceso al módulo          | Permitir ingresar al módulo de métodos de ordenamiento desde la página principal  | Alta      | Estudiante |
| RF-O-002 | Crear arreglo             | Permitir crear un arreglo inicial de valores                                      | Alta      | Estudiante |
| RF-O-003 | Validar arreglo           | Validar que los valores ingresados sean compatibles con los algoritmos soportados | Alta      | Estudiante |
| RF-O-004 | Generar arreglo aleatorio | Permitir generar un arreglo aleatorio pequeño o mediano para práctica             | Media     | Estudiante |
| RF-O-005 | Reiniciar arreglo         | Permitir restaurar el arreglo inicial o limpiar el estado                         | Alta      | Estudiante |
| RF-O-006 | Seleccionar algoritmo     | Permitir seleccionar un método de ordenamiento disponible                         | Alta      | Estudiante |
| RF-O-007 | Ejecutar modo rápido      | Ejecutar el algoritmo y mostrar directamente el resultado final                   | Alta      | Estudiante |
| RF-O-008 | Ejecutar paso a paso      | Permitir avanzar y retroceder por la traza de ejecución                           | Alta      | Estudiante |
| RF-O-009 | Resaltar comparación      | Mostrar visualmente los elementos que se están comparando                         | Alta      | Estudiante |
| RF-O-010 | Resaltar intercambio      | Mostrar visualmente cuando dos elementos cambian de posición                      | Alta      | Estudiante |
| RF-O-011 | Resaltar subarreglos      | Mostrar rangos activos en algoritmos recursivos o por partición                   | Media     | Estudiante |

| ID       | Requisito                | Descripción                                                                | Prioridad | Actor         |
|----------|--------------------------|----------------------------------------------------------------------------|-----------|---------------|
| RF-O-012 | Mostrar pivote           | Mostrar pivote en Quick Sort si el algoritmo existe                        | Media     | Estudiante    |
| RF-O-013 | Mostrar fusión           | Mostrar proceso de mezcla en Merge Sort si el algoritmo existe             | Media     | Estudiante    |
| RF-O-014 | Mostrar montículo        | Mostrar relación con Heap Sort si el algoritmo existe                      | Media     | Estudiante    |
| RF-O-015 | Mostrar contadores       | Mostrar comparaciones, intercambios, movimientos y pasos                   | Alta      | Estudiante    |
| RF-O-016 | Mostrar estado final     | Mostrar arreglo ordenado y mensaje de finalización                         | Alta      | Estudiante    |
| RF-O-017 | Código C                 | Mostrar código C asociado al algoritmo cuando exista mapeo                 | Alta      | Estudiante    |
| RF-O-018 | Pseudocódigo fallback    | Mostrar pseudocódigo cuando no exista código C mapeado                     | Alta      | Estudiante    |
| RF-O-019 | Resaltado de línea C     | Sincronizar línea de código C con paso visual                              | Alta      | Estudiante    |
| RF-O-020 | Consola didáctica        | Mostrar mensajes equivalentes a salida de ejecución o explicación          | Media     | Estudiante    |
| RF-O-021 | Historial                | Registrar algoritmos ejecutados y arreglos usados en la sesión             | Media     | Estudiante    |
| RF-O-022 | Ayuda del módulo         | Proveer ayuda general sobre ordenamiento                                   | Alta      | Estudiante    |
| RF-O-023 | Ayuda por algoritmo      | Proveer explicación por método disponible                                  | Alta      | Estudiante    |
| RF-O-024 | Manejo de errores        | Traducir errores técnicos a mensajes didácticos                            | Alta      | Estudiante    |
| RF-O-025 | Estado de sesión         | Mantener temporalmente el arreglo, algoritmo y traza activa                | Media     | Estudiante    |
| RF-O-026 | Separación lógica/visual | Separar algoritmos, adaptadores, servicios, rutas, plantillas y JavaScript | Alta      | Desarrollador |
| RF-O-027 | Preparación para C       | Mantener contratos compatibles con implementaciones C                      | Alta      | Desarrollador |
| RF-O-028 | Plantillas reutilizables | Usar el mismo header, footer, navbar y layout base del proyecto            | Alta      | Desarrollador |

## 4. Requisitos no funcionales

| ID        | Tipo                 | Descripción                                                                                                                    |
|-----------|----------------------|--------------------------------------------------------------------------------------------------------------------------------|
| RNF-O-001 | Documentación        | Todo el código debe estar documentado según estándares del lenguaje                                                            |
| RNF-O-002 | Python               | El código Python debe cumplir PEP8, usar docstrings y type hints cuando sea razonable                                          |
| RNF-O-003 | C                    | El código C debe mantenerse como referencia didáctica y no ejecutarse directamente en servidor salvo decisión futura explícita |
| RNF-O-004 | JavaScript           | Separar eventos, renderizado, animaciones y comunicación con backend                                                           |
| RNF-O-005 | HTML                 | Usar HTML semántico y plantillas Jinja2 reutilizables                                                                          |
| RNF-O-006 | CSS                  | Separar estilos base, layout, componentes y visualización de ordenamiento                                                      |
| RNF-O-007 | Modularidad          | Cada algoritmo debe estar encapsulado mediante adaptador o estrategia                                                          |
| RNF-O-008 | Mantenibilidad       | Debe ser sencillo agregar nuevos algoritmos                                                                                    |
| RNF-O-009 | Seguridad            | Validar entradas y evitar ejecución de código arbitrario                                                                       |
| RNF-O-010 | Usabilidad           | La interfaz debe ser clara para estudiantes principiantes                                                                      |
| RNF-O-011 | Rendimiento          | El sistema debe funcionar con arreglos pequeños y medianos de uso didáctico                                                    |
| RNF-O-012 | Extensibilidad       | La integración futura con C no debe afectar la interfaz web                                                                    |
| RNF-O-013 | Trazabilidad         | Cada requisito debe mapearse con casos de uso, módulos y pruebas                                                               |
| RNF-O-014 | Accesibilidad        | Usar colores, etiquetas y textos alternativos para reforzar comprensión                                                        |
| RNF-O-015 | Consistencia visual  | Mantener el mismo sistema visual de los módulos anteriores                                                                     |
| RNF-O-016 | Claridad algorítmica | Cada paso debe explicar qué condición, comparación o intercambio ocurre                                                        |
| RNF-O-017 | Tamaño didáctico     | Limitar el tamaño del arreglo para evitar animaciones lentas o confusas                                                        |

## 5. Casos de uso

### CU-O-001 — Acceder al módulo de ordenamiento

**Actor:** Estudiante

**Descripción:** El usuario ingresa al módulo de métodos de ordenamiento.

**Precondiciones:** La aplicación principal está disponible.

**Flujo principal:**

1. El usuario abre la página principal.
2. Selecciona "Métodos de ordenamiento".
3. El sistema muestra la página principal del módulo.
4. El sistema muestra una explicación breve y controles iniciales.

**Postcondición:** El usuario puede crear un arreglo y seleccionar un algoritmo.

---

### CU-O-002 — Crear arreglo manual

**Actor:** Estudiante

**Descripción:** El usuario ingresa manualmente los valores del arreglo.

**Precondiciones:** El usuario está en el módulo.

**Flujo principal:**

1. El usuario ingresa valores separados por coma o espacio.
2. El sistema valida los valores.
3. El sistema crea el arreglo en sesión.
4. El sistema muestra el arreglo en la visualización.

**Flujo alternativo:** Si hay valores inválidos, se muestra mensaje didáctico.

**Postcondición:** Existe un arreglo activo.

---

### CU-O-003 — Generar arreglo aleatorio

**Actor:** Estudiante

**Descripción:** El usuario genera un arreglo aleatorio para práctica.

**Precondiciones:** El usuario está en el módulo.

**Flujo principal:**

1. El usuario define tamaño y rango de valores.
2. El sistema valida los parámetros.
3. El sistema genera el arreglo.
4. El sistema muestra el arreglo en pantalla.

**Postcondición:** Existe un arreglo activo generado por el sistema.

---

## CU-O-004 — Seleccionar algoritmo

**Actor:** Estudiante

**Descripción:** El usuario selecciona un método de ordenamiento.

**Precondiciones:** Existe un arreglo activo.

**Flujo principal:**

1. El usuario abre la lista de algoritmos disponibles.
2. Selecciona un algoritmo.
3. El sistema carga descripción, ayuda, código C o pseudocódigo.
4. El sistema habilita la ejecución.

**Postcondición:** Existe un algoritmo activo.

---

## CU-O-005 — Ejecutar ordenamiento rápido

**Actor:** Estudiante

**Descripción:** El usuario ejecuta el algoritmo sin animación paso a paso.

**Precondiciones:** Existe arreglo y algoritmo activo.

**Flujo principal:**

1. El usuario presiona "Ejecutar".
2. El sistema valida el estado.
3. El algoritmo se ejecuta.
4. El sistema muestra el arreglo ordenado.
5. El sistema muestra contadores y mensaje final.

**Postcondición:** El usuario visualiza el resultado final.

---

## CU-O-006 — Ejecutar ordenamiento paso a paso

**Actor:** Estudiante

**Descripción:** El usuario ejecuta el algoritmo mediante traza interpretada.

**Precondiciones:** Existe arreglo y algoritmo activo.

**Flujo principal:**

1. El usuario activa "Interpretar paso a paso".
2. Presiona "Ejecutar".
3. El sistema genera la traza completa.
4. El sistema muestra el primer paso.
5. El usuario avanza o retrocede entre pasos.
6. El sistema sincroniza visualización, explicación y línea de código.

**Postcondición:** El usuario comprende el proceso completo.

---

## CU-O-007 — Consultar ayuda

**Actor:** Estudiante

**Descripción:** El usuario consulta ayuda del módulo o de un algoritmo.

**Precondiciones:** El usuario está en la aplicación.

**Flujo principal:**

1. El usuario selecciona ayuda.
2. El sistema muestra definición conceptual.
3. El sistema muestra funcionamiento, complejidad, ventajas y limitaciones.
4. El sistema muestra errores comunes.

**Postcondición:** El usuario obtiene apoyo didáctico.

---

## 6. Modelo de datos

### 6.1 Diccionario de datos

**Entidad: SortingSession**

| Atributo           | Tipo            | Descripción                                      |
|--------------------|-----------------|--------------------------------------------------|
| session_id         | string          | Identificador de sesión                          |
| module             | string          | Valor fijo: sorting                              |
| original_array     | list[int/float] | Arreglo inicial                                  |
| current_array      | list[int/float] | Arreglo actual                                   |
| selected_algorithm | string          | Algoritmo activo                                 |
| execution_trace    | list            | Pasos de ejecución generados                     |
| current_step       | int             | Índice del paso actual                           |
| metrics            | dict            | Comparaciones, intercambios, movimientos y pasos |
| last_operation     | string          | Última operación ejecutada                       |
| last_result        | dict/string     | Resultado de la última operación                 |
| errors             | list            | Errores recientes                                |

**Entidad: SortingAlgorithmDefinition**

| Atributo    | Tipo      | Descripción                                      |
|-------------|-----------|--------------------------------------------------|
| id          | string    | Identificador interno                            |
| name        | string    | Nombre visible                                   |
| description | string    | Descripción didáctica                            |
| stable      | bool/null | Indica si el algoritmo es estable cuando aplique |



| Atributo           | Tipo      | Descripción                          |
|--------------------|-----------|--------------------------------------|
| in_place           | bool/null | Indica si ordena en el mismo arreglo |
| complexity_best    | string    | Mejor caso                           |
| complexity_average | string    | Caso promedio                        |
| complexity_worst   | string    | Peor caso                            |
| space_complexity   | string    | Complejidad espacial                 |
| operations         | list      | Operaciones o fases visuales         |
| c_code_mapping     | dict/null | Mapeo a código C si existe           |
| help_content       | dict      | Ayuda asociada                       |

#### Entidad: SortingVisualState

| Atributo            | Tipo      | Descripción                      |
|---------------------|-----------|----------------------------------|
| structure           | string    | Valor fijo: sorting_array        |
| algorithm           | string    | Algoritmo activo                 |
| items               | list      | Valores actuales                 |
| highlighted_indices | list      | Índices resaltados               |
| comparing_indices   | list      | Índices en comparación           |
| swapping_indices    | list      | Índices en intercambio           |
| sorted_indices      | list      | Índices ya ordenados             |
| active_range        | list/null | Rango activo del algoritmo       |
| pivot_index         | int/null  | Índice pivote si aplica          |
| auxiliary_array     | list/null | Arreglo auxiliar si aplica       |
| metrics             | dict      | Contadores actuales              |
| last_operation      | dict      | Resultado de la última operación |

#### Entidad: SortingTraceStep

| Atributo          | Tipo     | Descripción                          |
|-------------------|----------|--------------------------------------|
| step              | int      | Número de paso                       |
| action            | string   | Acción didáctica del paso            |
| line              | int/null | Línea de código C asociada si existe |
| array_snapshot    | list     | Estado del arreglo en el paso        |
| comparing_indices | list     | Índices comparados                   |

| Atributo           | Tipo      | Descripción                         |
|--------------------|-----------|-------------------------------------|
| swapping_indices   | list      | Índices intercambiados              |
| sorted_indices     | list      | Índices ordenados                   |
| active_range       | list/null | Rango activo                        |
| pivot_index        | int/null  | Pivote si aplica                    |
| auxiliary_snapshot | list/null | Estado auxiliar si aplica           |
| console_output     | string    | Mensaje didáctico o salida simulada |
| metrics            | dict      | Métricas acumuladas                 |

## 7. Arquitectura del sistema

### 7.1 Estilo arquitectónico

El módulo de ordenamiento debe seguir la misma arquitectura por capas del proyecto:

```

Usuario
  ↓
Frontend HTML/CSS/JavaScript
  ↓
Rutas Flask / Controladores
  ↓
Servicios de aplicación
  ↓
Adaptador / estrategias de ordenamiento
  ↓
TAD o implementación base de ordenamiento
  ↓
Código C didáctico / futura integración C

```

### 7.2 Capas principales

#### Capa de presentación

Responsable de:

- Renderizar plantillas del módulo.
- Dibujar el arreglo como barras o celdas.
- Animar comparaciones e intercambios.
- Mostrar subarreglos, pivotes o arreglos auxiliares.
- Mostrar código C/pseudocódigo.
- Mostrar ayuda y mensajes didácticos.

## Capa de rutas Flask

Responsable de:

- Recibir solicitudes HTTP.
- Validar datos básicos.
- Invocar servicios.
- Retornar HTML o JSON.

No debe contener lógica interna de algoritmos.

## Capa de servicios

Responsable de:

- Coordinar creación de arreglo.
- Ejecutar algoritmos.
- Generar trazas.
- Manejar estado de sesión.
- Capturar errores.
- Traducir respuestas a mensajes didácticos.
- Entregar estados serializables al frontend.

## Capa de adaptadores / estrategias

Responsable de normalizar el acceso a los algoritmos de ordenamiento.

Adaptadores previstos:

- SortingAdapter.
- SortingAlgorithmStrategy.
- BubbleSortStrategy, si existe en el TAD.
- SelectionSortStrategy, si existe en el TAD.
- InsertionSortStrategy, si existe en el TAD.
- ShellSortStrategy, si existe en el TAD.
- MergeSortStrategy, si existe en el TAD.
- QuickSortStrategy, si existe en el TAD.
- HeapSortStrategy, si existe en el TAD.

## Capa de dominio

Contiene o referencia:

- `tad_ordenamiento.c`.
- `tad_ordenamiento.h`.
- Implementaciones Python equivalentes solo si se requieren para simular estado visual sin compilar C.

La capa de dominio debe permanecer independiente de Flask, plantillas y JavaScript.

---

## 8. Estructura de carpetas propuesta

```
visualizador_estructuras/  
├── app/  
│   ├── routes/  
│   │   ├── sorting_routes.py  
│   │   └── help_routes.py  
│   ├── services/  
│   │   ├── sorting_service.py  
│   │   ├── sorting_trace_service.py  
│   │   ├── session_service.py  
│   │   ├── c_code_service.py  
│   │   ├── pseudocode_service.py  
│   │   └── help_service.py  
│   ├── adapters/  
│   │   ├── base_adapter.py  
│   │   └── sorting_adapter.py  
│   ├── domain/  
│   │   ├── sorting/  
│   │   │   ├── __init__.py  
│   │   │   ├── tad_ordenamiento.c  
│   │   │   ├── tad_ordenamiento.h  
│   │   │   └── sorting_algorithms.py  
│   ├── templates/  
│   │   ├── base.html  
│   │   ├── partials/  
│   │   │   ├── header.html  
│   │   │   ├── footer.html  
│   │   │   ├── navbar.html  
│   │   │   ├── messages.html  
│   │   │   └── help_panel.html  
│   ├── pages/  
│   │   ├── sorting.html  
│   │   └── sorting_view.html  
│   └── help/  
│       ├── sorting_help.html  
│       ├── bubble_sort_help.html  
│       ├── selection_sort_help.html  
│       ├── insertion_sort_help.html  
│       ├── shell_sort_help.html  
│       ├── merge_sort_help.html  
│       └── quick_sort_help.html
```

```

├── heap_sort_help.html
├── static/
│   ├── css/
│   │   ├── sorting.css
│   │   └── sorting_visualizer.css
│   ├── js/
│   │   ├── sorting_api_client.js
│   │   ├── sorting_visualizer.js
│   │   ├── sorting_animations.js
│   │   ├── sorting_stepper.js
│   │   └── sorting_helpers.js
│   └── img/
├── tests/
│   ├── test_sorting_adapter.py
│   ├── test_sorting_routes.py
│   ├── test_sorting_visual_state.py
│   ├── test_sorting_trace.py
│   ├── test_sorting_algorithms.py
│   └── test_sorting_help.py
└── docs/
    ├── sorting_sdd.md
    ├── sorting_user_guide.md
    └── future_c_integration.md

```

## 9. Contratos de interfaz UI/UX

### Pantalla del módulo de ordenamiento

Debe incluir:

- Título: "Métodos de Ordenamiento".
- Descripción general.
- Campo para ingresar arreglo manual.
- Botón para generar arreglo aleatorio.
- Selector de algoritmo.
- Selector de modo:
  - Rápido.
  - Paso a paso.
- Botón "Ejecutar".
- Botón "Reiniciar".
- Botón "Ayuda del módulo".

## Pantalla de visualización de ordenamiento

|                                             |                              |
|---------------------------------------------|------------------------------|
| Header / Navbar                             |                              |
| Nombre del módulo                           |                              |
| Algoritmo activo y métricas                 |                              |
| Panel de operaciones<br>arreglo + algoritmo | Visualización<br>del arreglo |
| Código C / Pseudocódigo                     |                              |
| Explicación del paso actual                 |                              |
| Consola / historial                         |                              |
| Controles paso a paso                       |                              |
| Footer                                      |                              |

## Contrato visual para ordenamiento

```
{
  "structure": "sorting_array",
  "algorithm": "bubble_sort",
  "items": [5, 3, 8, 1],
  "highlighted_indices": [0, 1],
  "comparing_indices": [0, 1],
  "swapping_indices": [],
  "sorted_indices": [],
  "active_range": [0, 3],
  "pivot_index": null,
  "auxiliary_array": null,
  "metrics": {
    "comparisons": 1,
    "swaps": 0,
    "moves": 0,
    "steps": 1
  },
  "last_operation": {
    "name": "compare",
    "status": "running",
    "message": "Se comparan los elementos en las posiciones 0 y 1."
  }
}
```

## Contrato para traza de ejecución

```
{
  "algorithm": "bubble_sort",
  "current_step": 2,
  "total_steps": 12,
  "trace": [
    {
      "step": 1,
      "line": 8,
      "action": "Comparar arreglo[0] y arreglo[1]",
      "array_snapshot": [5, 3, 8, 1],
      "comparing_indices": [0, 1],
      "swapping_indices": [],
      "sorted_indices": [],
      "console_output": "Comparando 5 y 3",
      "metrics": {
        "comparisons": 1,
        "swaps": 0,
        "moves": 0
      }
    },
    {
      "step": 2,
      "line": 10,
      "action": "Intercambiar 5 y 3 porque están en orden incorrecto",
      "array_snapshot": [3, 5, 8, 1],
      "comparing_indices": [],
      "swapping_indices": [0, 1],
      "sorted_indices": [],
      "console_output": "Intercambio realizado",
      "metrics": {
        "comparisons": 1,
        "swaps": 1,
        "moves": 0
      }
    }
  ]
}
```

## 10. Reglas de negocio

| ID       | Regla             | Descripción                                          |
|----------|-------------------|------------------------------------------------------|
| RB-O-001 | Arreglo requerido | No se puede ejecutar un algoritmo sin arreglo activo |

| ID       | Regla                    | Descripción                                                                                     |
|----------|--------------------------|-------------------------------------------------------------------------------------------------|
| RB-O-002 | Algoritmo requerido      | No se puede ordenar sin algoritmo seleccionado                                                  |
| RB-O-003 | Valores válidos          | Los valores del arreglo deben cumplir el tipo soportado, inicialmente enteros                   |
| RB-O-004 | Tamaño mínimo            | El arreglo debe tener al menos un elemento para visualizarse                                    |
| RB-O-005 | Tamaño máximo didáctico  | El tamaño debe limitarse para asegurar claridad visual                                          |
| RB-O-006 | Modo rápido equivalente  | El resultado final del modo rápido debe ser igual al último paso del modo interpretado          |
| RB-O-007 | No inventar algoritmos   | Si un algoritmo no existe en el TAD C o implementación disponible, debe marcarse como pendiente |
| RB-O-008 | Código C como referencia | El servidor no debe compilar ni ejecutar C salvo decisión futura explícita                      |
| RB-O-009 | Pseudocódigo fallback    | Si no hay mapeo C, usar pseudocódigo claramente marcado                                         |
| RB-O-010 | Estado serializable      | El backend debe entregar estados JSON serializables                                             |
| RB-O-011 | Dominio independiente    | El dominio no debe importar Flask ni depender de la interfaz                                    |
| RB-O-012 | Contrato estable         | Los nombres internos de algoritmos deben mantenerse estables                                    |
| RB-O-013 | Integración futura con C | El cambio a ejecución C real debe ocurrir detrás del adaptador                                  |
| RB-O-014 | Validación de entradas   | Todo arreglo, tamaño, rango y algoritmo debe validarse antes de ejecutar                        |
| RB-O-015 | Trazas didácticas        | Cada paso debe tener acción, estado y explicación comprensible                                  |

## 11. Operaciones iniciales del módulo

### Operaciones generales

- Crear arreglo manual.
- Generar arreglo aleatorio.
- Seleccionar algoritmo.
- Ejecutar ordenamiento rápido.
- Ejecutar ordenamiento paso a paso.
- Avanzar paso.



- Retroceder paso.
- Reiniciar ejecución.
- Restaurar arreglo original.
- Limpiar arreglo.
- Consultar métricas.
- Consultar ayuda.

## Métodos de ordenamiento esperados

Los métodos concretos deben extraerse desde `tad_ordenamiento.c` y `tad_ordenamiento.h`.  
Como candidatos iniciales:

- Bubble Sort.
- Selection Sort.
- Insertion Sort.
- Shell Sort.
- Merge Sort.
- Quick Sort.
- Heap Sort.

## Observaciones didácticas

- Burbuja debe enfatizar comparaciones adyacentes e intercambios.
- Selección debe enfatizar búsqueda del mínimo o máximo.
- Inserción debe enfatizar desplazamientos.
- Shell debe enfatizar saltos o gaps.
- Merge debe enfatizar división y mezcla.
- Quick debe enfatizar pivote y partición.
- Heap debe enfatizar relación con montículos.

Si el TAD real no contiene alguno de estos algoritmos, debe marcarse como pendiente.

---

# 12. Especificación de algoritmos

## Algoritmo general para ejecutar ordenamiento

### Entrada:

- Arreglo actual.
- Algoritmo seleccionado.
- Modo de ejecución.
- Estado actual de sesión.

### Proceso:

1. Validar que exista arreglo activo.
2. Validar que exista algoritmo seleccionado.
3. Validar que el algoritmo esté soportado.
4. Generar traza completa de ejecución.
5. Si el modo es rápido, retornar el último estado.

6. Si el modo es paso a paso, retornar la traza desde el paso inicial.
7. Sincronizar traza con código C o pseudocódigo.
8. Actualizar métricas.
9. Guardar historial de sesión.
10. Retornar estado visual serializable.

**Salida:**

- Arreglo ordenado o paso actual.
- Traza de ejecución.
- Métricas.
- Mensaje didáctico.

### **Algoritmo para renderizar arreglo**

**Entrada:** Estado visual serializado.

**Proceso:**

1. Limpiar contenedor visual.
2. Crear una barra, celda o tarjeta por elemento.
3. Escalar altura si se usa representación de barras.
4. Aplicar color normal a elementos no activos.
5. Resaltar elementos comparados.
6. Resaltar elementos intercambiados.
7. Resaltar índices ordenados.
8. Mostrar pivote, rango activo o arreglo auxiliar si aplica.
9. Mostrar métricas actualizadas.

**Salida:** Arreglo visual actualizado.

### **Algoritmo para avanzar paso**

**Entrada:** Índice de paso actual.

**Proceso:**

1. Verificar que existe paso siguiente.
2. Incrementar índice.
3. Cargar snapshot del nuevo paso.
4. Actualizar visualización.
5. Resaltar línea de código asociada.
6. Actualizar consola y explicación.

**Salida:** Nuevo paso renderizado.

### **Algoritmo para retroceder paso**

**Entrada:** Índice de paso actual.

**Proceso:**

1. Verificar que existe paso anterior.
2. Disminuir índice.
3. Cargar snapshot del paso anterior.
4. Actualizar visualización.
5. Resaltar línea de código asociada.
6. Actualizar consola y explicación.

**Salida:** Paso anterior renderizado.

## 13. API / servicios

### Rutas web sugeridas

| Endpoint                                              | Método | Descripción                                     |
|-------------------------------------------------------|--------|-------------------------------------------------|
| <code>/ordenamiento</code>                            | GET    | Página del módulo de métodos de ordenamiento    |
| <code>/ordenamiento/visualizador</code>               | GET    | Página de visualización de ordenamiento         |
| <code>/api/ordenamiento/create-array</code>           | POST   | Crear arreglo manual                            |
| <code>/api/ordenamiento/random-array</code>           | POST   | Generar arreglo aleatorio                       |
| <code>/api/ordenamiento/algorithm</code>              | POST   | Seleccionar algoritmo                           |
| <code>/api/ordenamiento/run</code>                    | POST   | Ejecutar algoritmo en modo rápido o paso a paso |
| <code>/api/ordenamiento/step</code>                   | POST   | Avanzar o retroceder paso                       |
| <code>/api/ordenamiento/state</code>                  | GET    | Obtener estado actual serializado               |
| <code>/api/ordenamiento/reset</code>                  | POST   | Reiniciar ejecución o arreglo                   |
| <code>/ayuda/ordenamiento</code>                      | GET    | Ayuda general del módulo                        |
| <code>/ayuda/ordenamiento/&lt;algorithm_id&gt;</code> | GET    | Ayuda específica por algoritmo                  |

### Servicio: `SortingService`

Responsabilidades:

- Crear arreglo.
- Generar arreglo aleatorio.
- Seleccionar algoritmo.
- Ejecutar ordenamiento.
- Consultar estado.
- Reiniciar ejecución.
- Generar mensajes didácticos.

Métodos conceptuales:

```
create_array(values)
generate_random_array(size, min_value, max_value)
select_algorithm(algorithm_id)
run_sorting(mode)
get_sorting_state()
reset_sorting()
```

### **Servicio: SortingTraceService**

Responsabilidades:

- Generar trazas paso a paso.
- Registrar comparaciones.
- Registrar intercambios.
- Registrar movimientos.
- Asociar pasos con líneas de código C o pseudocódigo.
- Producir snapshots serializables.

### **Servicio: CCodeService**

Responsabilidades:

- Cargar código C del módulo.
- Mapear algoritmo a función C.
- Entregar estructura del TAD y función activa.
- Proveer líneas resaltables.

### **Servicio: PseudocodeService**

Responsabilidades:

- Proveer pseudocódigo cuando no exista mapeo C.
- Marcar claramente que se trata de fallback.

### **Servicio: HelpService**

Responsabilidades:

- Obtener ayuda del módulo.
- Obtener ayuda por algoritmo.
- Proveer complejidad, ventajas, desventajas y casos de uso.

### **Contrato de adaptador base**

Cada adaptador debe cumplir una interfaz común:

```
create()
execute(operation_name, payload)
```

```

to_visual_state()
reset()
get_supported_operations()

```

### Contrato de SortingAdapter

```

create_array(values)
generate_random_array(size, min_value, max_value)
select_algorithm(algorithm_id)
run(mode)
step(direction)
to_visual_state()
reset()
get_supported_operations()
get_supported_algorithms()

```

## 14. Trazabilidad

| RF       | Caso de uso           | Módulo                | Prueba                          |
|----------|-----------------------|-----------------------|---------------------------------|
| RF-O-001 | CU-O-001              | Sorting Routes        | test_sorting_module_loads       |
| RF-O-002 | CU-O-002              | SortingService        | test_create_array               |
| RF-O-003 | CU-O-002              | SortingService        | test_invalid_array_input        |
| RF-O-004 | CU-O-003              | SortingService        | test_generate_random_array      |
| RF-O-005 | CU-O-005,<br>CU-O-006 | SortingAdapter        | test_reset_array                |
| RF-O-006 | CU-O-004              | SortingAdapter        | test_select_algorithm           |
| RF-O-007 | CU-O-005              | SortingAdapter        | test_run_fast_mode              |
| RF-O-008 | CU-O-006              | SortingTraceService   | test_run_step_by_step           |
| RF-O-009 | CU-O-006              | Sorting Visualizer JS | test_comparison_highlight_state |
| RF-O-010 | CU-O-006              | Sorting Visualizer JS | test_swap_highlight_state       |

| RF       | Caso de uso            | Módulo                          | Prueba                                     |
|----------|------------------------|---------------------------------|--------------------------------------------|
| RF-O-011 | CU-O-006               | SortingTraceService             | test_active_range_state                    |
| RF-O-012 | CU-O-006               | SortingTraceService             | test_pivot_state_when_supported            |
| RF-O-013 | CU-O-006               | SortingTraceService             | test_merge_state_when_supported            |
| RF-O-014 | CU-O-006               | SortingTraceService             | test_heap_state_when_supported             |
| RF-O-015 | CU-O-005,<br>CU-O-006  | SortingTraceService             | test_sorting_metrics                       |
| RF-O-016 | CU-O-005,<br>CU-O-006  | SortingAdapter                  | test_sorted_final_state                    |
| RF-O-017 | CU-O-004,<br>CU-O-006  | CCodeService                    | test_sorting_c_code_mapping                |
| RF-O-018 | CU-O-004,<br>CU-O-006  | PseudocodeService               | test_pseudocode_fallback                   |
| RF-O-019 | CU-O-006               | SortingTraceService             | test_active_c_line                         |
| RF-O-020 | CU-O-006               | SortingTraceService             | test_console_output                        |
| RF-O-021 | CU-O-005,<br>CU-O-006  | SessionService                  | test_sorting_history                       |
| RF-O-022 | CU-O-007               | HelpService                     | test_sorting_help_available                |
| RF-O-023 | CU-O-007               | HelpService                     | test_algorithm_help_available              |
| RF-O-024 | CU-O-002 a<br>CU-O-006 | Services / Exceptions           | test_sorting_error_messages                |
| RF-O-025 | CU-O-002 a<br>CU-O-006 | SessionService                  | test_sorting_session_persistence           |
| RF-O-026 | Todos                  | Domain / Services /<br>Adapters | test_no_flask_dependency_in_sorting_domain |
| RF-O-027 | Futuro                 | Adapter Layer                   | test_sorting_adapter_contract_stability    |
| RF-O-028 | Todos                  | Templates                       | test_base_template_reuse                   |

## 15. Plan de pruebas

### Pruebas unitarias

Deben cubrir:

- SortingAdapter.
- SortingService.
- SortingTraceService.
- CCodeService.
- PseudocodeService.
- Validaciones de entrada.
- Generación de trazas.
- Métricas.
- Conversión a estado visual.

### Pruebas de integración

Deben validar:

- Rutas Flask del módulo.
- Creación de arreglo.
- Selección de algoritmo.
- Ejecución rápida.
- Ejecución paso a paso.
- Avance y retroceso de pasos.
- Respuesta JSON/HTML.
- Persistencia temporal en sesión.

### Pruebas de interfaz

Deben validar:

- Carga de plantillas.
- Botones visibles.
- Visualización del arreglo.
- Resaltado de comparaciones.
- Resaltado de intercambios.
- Visualización de métricas.
- Visualización de código C o pseudocódigo.
- Controles paso a paso.

### Pruebas didácticas

Deben validar que:

- Cada algoritmo tenga explicación.
- Cada paso tenga mensaje comprensible.
- Las comparaciones sean visibles.
- Los intercambios sean visibles.
- El modo rápido termine igual que el último paso del modo interpretado.

- Los errores sean comprensibles.
- 

## 16. Criterios de aceptación

La primera versión del módulo de métodos de ordenamiento será aceptada cuando:

1. La aplicación Flask mantenga la navegación general.
  2. Exista una página del módulo de métodos de ordenamiento.
  3. Se pueda crear un arreglo manual válido.
  4. Se pueda generar un arreglo aleatorio didáctico.
  5. Se bloquee un arreglo inválido y se explique el error.
  6. Se pueda seleccionar un algoritmo disponible.
  7. Se pueda ejecutar ordenamiento en modo rápido.
  8. Se pueda ejecutar ordenamiento paso a paso.
  9. Se puedan avanzar y retroceder pasos.
  10. La visualización muestre comparaciones.
  11. La visualización muestre intercambios o movimientos.
  12. La visualización muestre el arreglo final ordenado.
  13. Se muestren métricas de ejecución.
  14. Se muestre código C cuando exista mapeo.
  15. Se muestre pseudocódigo cuando no exista mapeo C.
  16. El modo rápido produzca el mismo estado final que el modo paso a paso.
  17. Los errores sean controlados y explicados de forma didáctica.
  18. El código de dominio no importe Flask.
  19. Las plantillas reutilicen header, footer y navbar del proyecto.
  20. Exista ayuda didáctica del módulo y de cada algoritmo.
  21. El código Python cumpla PEP8 y tenga docstrings.
  22. El frontend esté separado en archivos CSS y JavaScript.
  23. Existan pruebas iniciales del módulo.
  24. El proyecto quede preparado para futuras implementaciones de TAD en C.
- 

## 17. Gestión del proyecto

### Metodología

Desarrollo basado en especificaciones, con implementación incremental por algoritmo y trazas didácticas.

### Herramientas sugeridas

- Python 3.
- Flask.
- Pytest.
- HTML5.
- CSS3.
- JavaScript.
- C como referencia didáctica.
- Git.



- Agente IA de desarrollo.

## Fases

### Fase 1 — Análisis del TAD de ordenamiento

- Revisar `tad_ordenamiento.c`.
- Revisar `tad_ordenamiento.h`.
- Extraer funciones, firmas y algoritmos reales.
- Identificar funciones públicas.
- Identificar limitaciones de visualización.
- Identificar mapeos de código C por algoritmo.

### Fase 2 — Base del módulo

- Crear rutas de ordenamiento.
- Crear plantillas del módulo.
- Integrar navegación desde la página principal.

### Fase 3 — Adaptador y estrategias

- Crear `SortingAdapter`.
- Crear estrategias por algoritmo soportado.
- Normalizar operaciones.
- Crear estado visual serializable.

### Fase 4 — Trazas didácticas

- Crear `SortingTraceService`.
- Generar pasos de comparación.
- Generar pasos de intercambio.
- Generar pasos de rango activo, pivote o arreglo auxiliar cuando aplique.
- Sincronizar con código C o pseudocódigo.

### Fase 5 — Visualización

- Crear visualizador de arreglo.
- Animar comparaciones e intercambios.
- Mostrar contadores.
- Mostrar código y consola.
- Crear controles paso a paso.

### Fase 6 — Ayuda didáctica

- Crear ayuda general del módulo.
- Crear ayuda por algoritmo.
- Incluir complejidad, estabilidad, uso de memoria y casos de uso.

### Fase 7 — Pruebas y documentación

- Agregar pruebas unitarias.
- Agregar pruebas de rutas.
- Agregar pruebas de trazas.

- Documentar arquitectura.
- Documentar uso del módulo.
- Documentar preparación para C.

## 18. Integración con agentes de IA

### 18.1 Roles

| Agente            | Responsabilidad                                                     |
|-------------------|---------------------------------------------------------------------|
| Agente arquitecto | Mantener arquitectura, trazabilidad y separación de capas           |
| Agente backend    | Implementar Flask, servicios, adaptadores, sesión y pruebas backend |
| Agente frontend   | Implementar plantillas, CSS, JS y visualización de ordenamiento     |
| Agente documental | Crear ayudas didácticas, README, guías y docstrings                 |
| Agente QA         | Crear pruebas unitarias, integración y validación de aceptación     |

### 18.2 Prompt base para agente desarrollador

Actúa como un agente desarrollador senior especializado en Python, Flask, HTML5, CSS3, JavaScript, C didáctico y arquitectura limpia para aplicaciones educativas.

Contexto del proyecto:

Debes construir un nuevo módulo de una aplicación web didáctica llamada "Visualizador Web de Estructuras de Datos". El objetivo del módulo es permitir que estudiantes aprendan métodos de ordenamiento mediante visualizaciones interactivas, ejecución paso a paso y sincronización con código C o pseudocódigo.

La aplicación general tiene módulos de estructuras secuenciales, estructuras jerárquicas, grafos, tablas hash y ahora métodos de ordenamiento.

En esta fase solo debes implementar el módulo de métodos de ordenamiento.

Archivos disponibles inicialmente para este módulo:

- tad\_ordenamiento.c
- tad\_ordenamiento.h

Objetivo técnico:

Crear un módulo Flask modular que permita crear un arreglo, seleccionar un algoritmo de ordenamiento, ejecutar el algoritmo en modo rápido o paso a paso, visualizar comparaciones, intercambios, rangos activos, métricas y mostrar código C o pseudocódigo sincronizado. La lógica de algoritmos debe estar completamente separada de la capa visual.

Restricciones obligatorias:

1. No mezclar lógica de ordenamiento con rutas Flask ni plantillas HTML.
2. No modificar los archivos C originales salvo que sea estrictamente necesario y documentado.
3. No inventar algoritmos que no existan en los archivos suministrados. Si un algoritmo requerido no existe, marcarlo como pendiente.
4. Todo código Python debe cumplir PEP8.
5. Todo módulo, clase y función Python debe tener docstrings.
6. Usar nombres claros y type hints cuando sea razonable.
7. No usar eval.
8. Validar entradas de usuario.
9. Todos los errores deben traducirse a mensajes didácticos.
10. La interfaz debe usar plantillas reutilizables con header, footer, navbar y layout base.
11. El frontend debe estar separado en HTML, CSS y JavaScript.
12. La arquitectura debe quedar preparada para que en una fase futura los algoritmos puedan integrarse con C real.
13. No compilar ni ejecutar código C en el servidor salvo que el proyecto lo defina explícitamente en una fase posterior.
14. No exponer directamente objetos internos no serializables al frontend.
15. El modo rápido debe producir el mismo estado final que el último paso del modo interpretado.

Arquitectura esperada:

Implementa una arquitectura por capas:

- Capa de presentación:
  - templates/
  - static/css/
  - static/js/
- Capa de rutas:
  - app/routes/sorting\_routes.py
  - app/routes/help\_routes.py
- Capa de servicios:
  - app/services/sorting\_service.py
  - app/services/sorting\_trace\_service.py
  - app/services/session\_service.py
  - app/services/c\_code\_service.py
  - app/services/pseudocode\_service.py
  - app/services/help\_service.py
- Capa de adaptadores:
  - app/adapters/base\_adapter.py
  - app/adapters/sorting\_adapter.py
- Capa de dominio:
  - app/domain/sorting/
  - Allí deben ubicarse tad\_ordenamiento.c y tad\_ordenamiento.h.

Funcionalidades mínimas:

1. Página del módulo de métodos de ordenamiento.
2. Navegación desde la página principal.
3. Crear arreglo manual.
4. Generar arreglo aleatorio.
5. Seleccionar algoritmo disponible.
6. Ejecutar en modo rápido.
7. Ejecutar paso a paso.
8. Avanzar y retroceder pasos.
9. Visualizar arreglo como barras o celdas.
10. Resaltar comparaciones.
11. Resaltar intercambios o movimientos.
12. Mostrar rangos activos cuando aplique.
13. Mostrar pivote cuando aplique.
14. Mostrar arreglo auxiliar cuando aplique.
15. Mostrar métricas: comparaciones, intercambios, movimientos y pasos.
16. Mostrar código C cuando exista mapeo.
17. Mostrar pseudocódigo como fallback.
18. Mostrar mensajes didácticos después de cada acción.
19. Manejo de errores para arreglo inválido, algoritmo inexistente o ejecución inválida.
20. Sección de ayuda general para ordenamiento.
21. Ayuda específica para cada algoritmo disponible.

Contrato de adaptador:

Cada adaptador debe implementar una interfaz común equivalente a:

- create()
- execute(operation\_name, payload)
- to\_visual\_state()
- reset()
- get\_supported\_operations()

El SortingAdapter debe ofrecer también:

- create\_array(values)
- generate\_random\_array(size, min\_value, max\_value)
- select\_algorithm(algorithm\_id)
- run(mode)
- step(direction)
- get\_supported\_algorithms()

El método to\_visual\_state() debe devolver datos serializables, por ejemplo:

```
{
  "structure": "sorting_array",
  "algorithm": "bubble_sort",
  "items": [5, 3, 8, 1],
  "highlighted_indices": [0, 1],
  "comparing_indices": [0, 1],
  "swapping_indices": [],
  "sorted_indices": [],
```

```

    "metrics": {
      "comparisons": 1,
      "swaps": 0,
      "moves": 0,
      "steps": 1
    },
    "last_operation": {
      "name": "compare",
      "status": "running",
      "message": "Se comparan los elementos en las posiciones 0 y 1."
    }
  }
}

```

Pruebas requeridas:

Crea pruebas para:

1. Carga del módulo de ordenamiento.
2. Creación de arreglo válido.
3. Bloqueo de arreglo inválido.
4. Generación de arreglo aleatorio.
5. Selección de algoritmo válido.
6. Bloqueo de algoritmo inexistente.
7. Ejecución rápida.
8. Ejecución paso a paso.
9. Avance y retroceso de pasos.
10. Conversión a estado visual.
11. Resaltado de comparaciones.
12. Resaltado de intercambios.
13. Conteo de métricas.
14. Equivalencia entre modo rápido y último paso interpretado.
15. Mapeo de código C cuando exista.
16. Fallback a pseudocódigo cuando no exista código C.
17. Persistencia temporal del estado en sesión.
18. Disponibilidad de ayuda didáctica.

Criterios de aceptación:

La tarea se considera completa si:

1. La aplicación Flask ejecuta sin errores.
2. El módulo de ordenamiento está disponible desde la navegación.
3. Se puede crear un arreglo válido.
4. Se puede seleccionar un algoritmo disponible.
5. Las operaciones disponibles se obtienen desde adaptadores.
6. El estado visual se actualiza después de cada paso.
7. Se visualizan comparaciones e intercambios.
8. Se muestran métricas de ejecución.
9. Se muestra código C o pseudocódigo.
10. El modo rápido y el modo paso a paso terminan en el mismo estado final.
11. El código de dominio no importa Flask.
12. Las plantillas reutilizan header, footer y navbar.
13. Existe ayuda didáctica del módulo y de cada algoritmo.
14. Existen pruebas iniciales.
15. El proyecto queda preparado para futura integración de algoritmos

implementados en C.

Prohibiciones explícitas:

- No generar código monolítico.
- No poner lógica de ordenamiento en las rutas Flask.
- No poner lógica de negocio en JavaScript si debe estar en backend.
- No usar eval.
- No ignorar excepciones.
- No inventar algoritmos inexistentes en los archivos C.
- No eliminar archivos C originales.
- No acoplar la UI directamente a una función concreta del dominio.
- No compilar ni ejecutar C en servidor sin especificación adicional.
- No depender de detalles internos no documentados sin registrarlo como decisión técnica temporal.

Formato de salida esperado:

1. Primero crea un plan de implementación detallado antes de desarrollar la aplicación.
2. El plan debe incluir:
  - análisis de `tad_ordenamiento.c` y `tad_ordenamiento.h`;
  - lista real de funciones y algoritmos disponibles;
  - mapa entre operaciones de UI y funciones reales;
  - estructura final de carpetas;
  - fases de implementación;
  - componentes backend, frontend, servicios, adaptadores y pruebas;
  - riesgos técnicos;
  - operaciones pendientes o no soportadas;
  - criterios de validación antes de codificar.
3. Después de presentar el plan, desarrolla la aplicación siguiendo ese plan.
4. Muestra el árbol de carpetas creado.
5. Explica brevemente la responsabilidad de cada carpeta.
6. Implementa los archivos necesarios.
7. Incluye comentarios y docstrings.
8. Incluye pruebas.
9. Incluye instrucciones para ejecutar:
  - creación de entorno virtual;
  - instalación de dependencias;
  - ejecución de Flask;
  - ejecución de pruebas.
10. Indica claramente cualquier algoritmo pendiente porque no exista en los archivos suministrados.

Antes de codificar:

Analiza los archivos suministrados y extrae sus funciones, firmas, algoritmos y restricciones reales. Usa esa información para construir el adaptador y la traza. Si existe discrepancia entre esta especificación y los archivos reales, respeta los archivos reales y documenta la diferencia.

## 19. Control de versiones

### Rama principal

```
main
```

Debe contener solo versiones estables.

### Rama de desarrollo

```
develop
```

Para integración de funcionalidades.

### Ramas sugeridas

```
feature/sorting-module  
feature/sorting-routes  
feature/sorting-adapter  
feature/sorting-trace-service  
feature/sorting-visualizer  
feature/sorting-c-code-mapping  
feature/sorting-help  
feature/sorting-tests
```

### Convención de commits

```
feat: agrega módulo de métodos de ordenamiento  
feat: agrega adaptador de ordenamiento  
feat: agrega visualizador de arreglos  
feat: agrega trazas paso a paso para ordenamiento  
feat: integra código C para algoritmos de ordenamiento  
fix: corrige validación de arreglo inválido  
docs: agrega ayuda didáctica para quick sort  
test: agrega pruebas para trazas de ordenamiento  
refactor: separa visualización de lógica de ordenamiento
```

---

## 20. Decisiones arquitectónicas importantes

| ID       | Decisión                       | Justificación                               |
|----------|--------------------------------|---------------------------------------------|
| DA-O-001 | Separar dominio y presentación | Evita acoplamiento entre algoritmos y Flask |

| ID       | Decisión                           | Justificación                                                          |
|----------|------------------------------------|------------------------------------------------------------------------|
| DA-O-002 | Usar adaptador                     | Permite normalizar algoritmos y preparar integración con C             |
| DA-O-003 | Usar servicio de trazas            | Permite simular paso a paso sin mezclar UI y lógica                    |
| DA-O-004 | Serializar estado visual           | Facilita comunicación backend/frontend                                 |
| DA-O-005 | Usar plantillas base               | Mantiene consistencia con módulos anteriores                           |
| DA-O-006 | Código C como referencia didáctica | Alinea el módulo con la visión del proyecto sin ejecutar C en servidor |
| DA-O-007 | Pseudocódigo fallback              | Permite continuidad didáctica cuando no exista mapeo C                 |
| DA-O-008 | Modo rápido equivalente            | Garantiza coherencia entre simulación y resultado final                |
| DA-O-009 | Marcar pendientes técnicos         | Evita que agentes IA inventen algoritmos o funciones                   |
| DA-O-010 | Preparar integración con C         | Permite cambiar implementación sin rediseñar la UI                     |